

Plant Care Tracker: Annotated Requirements

Josh Dunlap, Aaron Fuentes, Nathan Reeves
CISC 450 (Spring 2026)

1. **A user can add a new plant, specifying its common name, species, approximate age, and location within the home.** A new plant is inserted into the `plants` table, with foreign keys to `species` (for species and recommended watering interval) and `room` (which is itself a child of `location`). Age can be stored either as `approx_age` or as an exact `birthday` column.
2. **A user can view a list of all plants, optionally filtered by room or location.** The plants list joins `plants`, `species`, `room`, and `location` and accepts optional `roomId` or `locationId` filters in a `WHERE` clause.
3. **A user can record a watering event for a specific plant on a given date.** Each watering inserts a row into `watering_event` with the `plant_id` foreign key, the `watered_on` date, and optional `notes`.
4. **A user can view the full watering history for a specific plant, including dates and any notes recorded at the time of watering.** The plant detail page selects all rows from `watering_event` for the given `plant_id`, ordered by `watered_on` descending, and renders the date and notes columns directly.
5. **A user can view the last date a specific plant was watered and how many days ago that was.** A single query against `watering_event` returns `MAX(watered_on)` along with `CURRENT_DATE - MAX(watered_on)` for the days-ago value.
6. **A user can enter a date and view all plants scheduled to be watered on that date.** The `watering_schedule` table stores planned waterings as (`plant_id`, `scheduled_date`) rows; the schedule page joins this table against `plants` and filters by the chosen date.
7. **A user can view all plants that are overdue for watering, based on the recommended watering interval defined for each species.** The dashboard query computes each plant's most recent watering date and compares `CURRENT_DATE - last_watered_on` against `species.recomm_water_interval` to flag overdue plants.
8. **A user can log a fertilization event for a plant, including the fertilizer type used and the date applied.** Each fertilization inserts a row into `fertilization_event` with the `plant_id` foreign key, the freeform `fert_type`, and the `date_applied`.
9. **A user can log a repotting event for a plant, recording the date and the new pot size.** Each repotting inserts a row into `repotting_event` (capturing previous pot size, new pot size, and date) and updates `plants.pot_size` in the same transaction so the current pot stays in sync.

10. **A user can add a freeform health observation to a plant, which is stored with a timestamp and viewable as a log.** Each observation inserts a row into `health_observation` with the `plant_id` foreign key, the date the observation was made in `observation_date`, and the freeform text in `notes`.
11. **A user can view a care summary for a specific plant showing the last watering, last fertilization, last repotting, and most recent health observation all in one view.** The detail page issues four parallel `ORDER BY date DESC LIMIT 1` queries (one per event table) and renders the results as four summary tiles above the log forms.